DATA-DRIVEN DEVELOPMENT FRAMEWORK

An inverted, scenario-based data-first approach to building business applications

Developed by Talenciaglobal

ITEM	DETAILS
Document type	Development framework
Version	1.1 - Updated Framework
Scope	Workflow-driven business applications
Status	Phase 1 Pilot / Learning Sprint



1. Purpose

This document defines a repeatable, data-driven framework for building workflow-driven business applications. It will first be applied to a Quality Scorecard application and then extended to other suitable projects across the portfolio.

The framework deliberately avoids starting with detailed screens or a final Product Requirements Document (PRD). It begins with an Initial Product Scope, uses scenario-based datasets to discover and validate what the application must support, and progressively tightens behaviour, specification and testing.

Core sequence: Initial Product Scope -> Data Foundation -> Scenario-Based Datasets -> Working Build -> Test & Migration -> Complex Business Workflows -> Refined PRD -> Comprehensive Verification -> Tuning.

2. Background: why the approach has changed

Software development has evolved from exhaustive requirements-first methods to iterative specification, visual prototyping and, increasingly, AI-assisted generation. For the class of applications targeted in Phase 1, modern tools make it practical to invert the traditional sequence and learn from data much earlier.

Two changes make this practical today: AI-assisted application generation has dramatically reduced the effort required to create responsive working interfaces, and LLMs plus simple programmatic tools can generate realistic data at scale once the scenario model and data rules are well defined.



STAGE	APPROACH	LIMITATION / LEARNING
1. Requirements-led	Write an exhaustive scope before building.	Requirements are rarely fully known upfront; they are discovered incrementally.
2. Precision-led	Prioritise a few features per sprint and write precise specifications.	Better, but still specification-first and relatively slow to validate.
3. Visualisation-led	Prototype screens and flows, then specify and code.	Useful for experience design, but can over-invest in screens before the data and scenarios are understood.
4. Data-led (Phase 1)	Define the Initial Product Scope, model data, design scenario-based datasets and derive a working application.	Tests business understanding early and produces concrete evidence for later workflow and specification refinement.

3. Phase 1 boundaries and evolution

Phase 1 is intentionally narrow. It is designed for workflow-driven applications where the business domain is comparatively easy to understand. The purpose is to validate the data-driven development approach before introducing additional layers of engineering complexity.

AREA	PHASE 1	PHASE 2
Product starting point	Initial Product Scope	Initial Product Scope
Domain modelling	Lightweight / implicit for simple domains	Explicit domain-first modelling before data
Data	Primary engineering foundation	Follows domain foundation
Scenario-based datasets	Core mechanism	Core mechanism, enriched by domain models
Complex business workflows	Introduced in Cycle 3	Modelled more formally where required
Security & Governance	Outside pilot scope	Integrated as a foundation

4. Core principles

1. Initial Product Scope, not an upfront PRD. Begin with the problem, actors, expected outcomes, constraints and broad functional scope. The detailed PRD is refined after the team has learned from data and application behaviour.

2. Data first within Phase 1. For the current class of simple-domain applications, the data model is the primary engineering foundation after the Initial Product Scope.

3. Scenario-based datasets. Data is organised around meaningful business scenarios rather than undifferentiated volume. Each scenario represents something the application must be able to store, process, display or reject.

4. Scenario richness before volume. The high-value work is identifying diverse scenarios and edge cases. Scale is generated only after those scenarios are sound.

5. Realism includes bad data. Invalid, incomplete, inconsistent and migration-style data is introduced deliberately so that failure behaviour is discovered early.

6. Iterate even on data. Schemas, dictionaries and scenario datasets evolve in increments. Each increment can drive a new working release.

7. Complex behaviour is tightened in Cycle 3. The first build establishes the data-driven foundation. Complex workflows, state transitions, business rules, exceptions and alternate paths are then formalised in Cycle 3.

8. Refined PRD emerges from evidence. The detailed PRD is produced after learning from scenario datasets, the working application, migration behaviour and complex workflow analysis.

9. Tight at both ends. Acceptance criteria and business expectations are held firmly on one side, with comprehensive automated verification on the other. The generative build zone between them can iterate rapidly.

5. The framework at a glance

STAGE	KEY STEPS	KEY OUTPUT
Initial Product Scope	Problem; actors; expected outcomes; constraints; broad functional scope	Clear product intent without pretending that detailed requirements are complete
Cycle 1: Data Foundation	Database design; data dictionary; scenario-based dataset design; data generation; analytics/BI; first application build	Populated database, scenario catalogue, BI and working first build
Cycle 2: Test & Migration	Test design; buggy/invalid scenario datasets; migration strategy; migration execution; defect report	Evidence of behaviour under flawed and migrated data
Cycle 3: Business Behaviour, Specification & Testing	Complex workflows; business rules; state transitions; exceptions; refined PRD; acceptance suite; comprehensive suite	Tight behavioural specification and automated verification net
Application Tuning	Make the real application pass the agreed suites and NFR expectations	Production-quality application

6. Initial Product Scope

The framework begins with an Initial Product Scope. This is deliberately lighter than a PRD. It gives the team enough context to begin modelling the data without creating false precision.

ELEMENT	QUESTIONS
Problem	What business problem are we solving? Why does the application need to exist?
Actors	Who creates, consumes, reviews, approves, administers or analyses the information?
Expected outcomes	What measurable or observable outcomes should the application enable?
Constraints	What known business, technology, timing, migration or operating constraints exist?
Broad functional scope	What major capabilities are clearly in scope, without attempting detailed feature specification?

7. Cycle 1: Data Foundation

Cycle 1 converts the Initial Product Scope into a rich data foundation and a first working version of the application.

7.1 Database design from the Initial Product Scope

Use the Initial Product Scope to propose the first database design. AI can accelerate the first draft, but people strong in database design must validate the model, including keys, relationships, integrity and appropriate normalisation. The model is expected to evolve as scenarios expose missing concepts.

7.2 Define the data dictionary

For each field define data type, length/format, business rules, nullability, keys/identity and validation rules. The data dictionary becomes the contract used to generate, ingest and validate data.

7.3 Scenario-based dataset design

This is the central intellectual step in Phase 1. Identify the distinct business situations the application must support. A scenario is not merely a row pattern; it represents a meaningful state or situation in the business.

Design principle: Scenario richness first. Volume second.

SCENARIO CATEGORY	ILLUSTRATIVE EXAMPLES
Normal / happy path	Standard scorecard with all expected fields and valid relationships
Business variants	Client-specific scorecard; optional parameters; alternate scorecard structures
Lifecycle states	Draft; partially completed; submitted; historical or archived records
Boundary cases	Minimum/maximum values; unusual combinations; sparse but valid records
Invalid / flawed data	Missing references; invalid values; inconsistent formats; duplicate identities
Migration cases	Legacy records with missing, stale or partially mapped information

7.4 to 7.6 Generation, analytics and first build

7.4 Data generation and ingestion

Build a programmatic data-generation and ingestion utility, with AI assistance where useful, that creates records according to the data dictionary and scenario catalogue. Maintain master-child sequencing and referential integrity. Once scenario coverage is strong, multiply the scenarios to create the volume required for scale testing.

7.5 Data analytics and business intelligence

Use the populated database to explore analytics and executive-level insights. This tests whether the underlying data model supports not only transactions and workflows, but also the intelligence expected from the product.

7.6 First working application

Feed the schema, data contracts and representative scenario data into an AI-assisted application generation layer to produce the first working application. Specific tools may change over time; the framework is tool-independent. The objective is a rapid vertical slice on top of a realistic data foundation.

8. Cycle 2: Test and Migration

Cycle 2 deliberately stresses the application with flawed data and migration conditions.

8.1 Test design

Design tests around the rules and scenarios established in Cycle 1.

8.2 Buggy and invalid scenario datasets

Generate controlled variations containing missing, malformed, inconsistent, duplicate and referentially invalid data. Preserve traceability to the scenario and injected defect.

8.3 Data migration strategy

Define source-to-target mapping, transformation, cleansing, validation, rejection and reconciliation rules.

8.4 Migration execution

Run migrations against representative flawed datasets and observe failures across data, workflows and screens.

8.5 Migration and test summary

Record defects, affected scenarios, validated rules, production impact and required changes to the schema, dictionary, application or migration logic.

9. Cycle 3: Business Behaviour, Specification and Testing

Cycle 3 is where the framework moves beyond the initial data-driven build and explicitly tightens complex business behaviour. The application is now informed by real data scenarios and migration learning, making workflow analysis more concrete.

9.1 Model complex business workflows

Define multi-step workflows, actor responsibilities, approvals, dependencies and alternate paths that are not adequately expressed by the data model alone.

9.2 Define business rules and state transitions

Specify allowed transitions, preconditions, postconditions, permissions, sequencing, temporal rules and other behavioural invariants.

9.3 Define exceptions and alternate flows

Capture rejection paths, cancellations, retries, overrides, unavailable actors, partial completion and other exceptional conditions.

9.4 to 9.6 PRD and test suites

9.4 Generate the refined PRD

Consolidate the Initial Product Scope, data model, scenario catalogue, migration learning and business behaviour into the refined Product Requirements Document.

9.5 Generate the acceptance test suite

Create a focused, business-readable feature-level suite that stakeholders can review and sign off. Each acceptance case can be backed by many automated internal tests.

9.6 Generate the comprehensive test suite

Build the appropriate functional, integration, data, migration, smoke, sanity, non-functional and other automated tests. Optimise for risk and scenario coverage rather than raw test-count volume.

Cycle 3 outcome: complex behaviour is explicit, the refined PRD is evidence-based, and the application is surrounded by a strong automated verification net.

10. Product Development and Application Tuning

For suitable Phase 1 products, the end-to-end sequence is: Product research -> Initial Product Scope -> Cycles 1, 2 and 3 -> Product architecture -> Application development / refinement -> Application tuning.

Application tuning is the last-mile engineering work required to make the real application satisfy the agreed acceptance criteria, comprehensive automated suites and applicable non-functional expectations. Al can accelerate generation throughout the lifecycle, while experienced engineers provide judgement where ambiguity, architecture and complex failure modes require it.

11. Data Model Evolution

The data model may mature as the application's purpose and cross-domain usage grow.

STAGE	DESCRIPTION	
Relational	Natural starting point for structured, workflow-driven applications such as the Quality Scorecard.	
NoSQL	Useful where complete business artefacts are best preserved and retrieved as documents.	
Graph / knowledge graph	Useful where relationships and cross-domain information become central.	
Contextual vectors	Useful when semantic context, retrieval or similarity becomes important.	
	Hybrid	Combination of models selected according to how the data is used.

12. Team Model

Core data team: Strong analytical contributors handle database modelling, the data dictionary and scenario-based dataset design.

Build team: Engineers and interns generate and refine the application on top of the prepared data foundation.

Behaviour and tuning team: Experienced engineers help formalise complex workflows in Cycle 3 and handle architecture-sensitive and last-mile refinement.

During Phase 1, the deliberate hypothesis is that data complexity is the best starting point for the targeted applications. Complex workflow behaviour is not ignored; it is introduced explicitly in Cycle 3 after the team has a concrete data-driven application to reason about.

13. Pilot: Quality Scorecard Learning Sprint

The Quality Scorecard remains the first learning sprint because it is small, needed by the team and suitable for validating the Phase 1 hypothesis.

Pilot measurement should include:

- Time to first working build
- Quality and completeness of scenario catalogue
- Schema changes triggered by scenarios
- Missed business situations discovered during migration and Cycle 3
- Human corrections required to AI-generated artefacts
- Defects found before and after complex workflow modelling
- Test coverage of agreed business rules and high-risk scenarios
- Total human effort per iteration



ASPECT	DETAILS
Application	Quality Scorecard based on the defined quality framework parameters
Participants	Cross-functional group of approximately nine team members, each building independently
Effort	Approximately one to two days per person for the first Cycle 1 experiment
Method	Review the framework; define Initial Product Scope; obtain/validate schema; design scenario-based datasets; generate data; create a first build; record the experience
Expectation	Learning POC; rough results are acceptable
Iterations	Repeat and progressively add Cycles 2 and 3; consolidate learning after each round
Scale-up	Extend to the wider intern and trainee pool and suitable projects after the framework stabilises

14. Implementation Roadmap

PHASE	ACTIVITIES
1. Orientation	Share the framework, Quality Scorecard context and Phase 1 boundaries.
2. Pilot Sprint	Each participant completes Initial Product Scope and Cycle 1 independently and records the experience.
3. Review	Compare schemas, scenario catalogues, generated data, applications and learning; consolidate the strongest patterns.
4. Repeat	Run further iterations, adding Cycle 2 and then Cycle 3 until the approach is well understood.
5. Scale-up	Apply the approach to a wider set of simple-domain, workflow-driven applications.
6. Competency building	Turn repeated learning into formal capabilities in data-driven and scenario-driven development.
7. Phase 2 evolution	Add explicit domain-first modelling and Security & Governance foundations when the Phase 1 model is stable.

15. Phase 2 Evolution

Phase 2 will extend rather than replace the Phase 1 framework. For more complex products, the Domain will explicitly precede Data, and Security & Governance will be introduced as a foundational concern.

Phase 2 target sequence: Initial Product Scope -> Domain Foundation -> Security & Governance Foundation -> Data Foundation -> Scenario-Based Datasets -> Build -> Test & Migration -> Business Behaviour -> Refined PRD -> Verification -> Tuning.

PHASE 2 FOUNDATION	INDICATIVE COVERAGE
Domain Foundation	Domain concepts; entities; relationships; bounded contexts where useful; domain invariants; domain language
Security & Governance Foundation	Identity; authorization; data classification; privacy; audit; retention; encryption; compliance requirements as applicable

16. Considerations and Open Points

- Security and compliance are intentionally outside the Phase 1 pilot and will be incorporated in Phase 2.
- Explicit domain-first modelling is intentionally deferred in Phase 1 because the initial applications are expected to be easy on the domain; it becomes a prerequisite in Phase 2 for more complex domains.
- Complex business workflows are not ignored in Phase 1. They are deliberately formalised in Cycle 3 after the data-driven foundation and first build exist.

- Scenario-based dataset quality is a major constraint. Designing realistic, diverse scenarios requires strong understanding of the users, client context and business situations.
- Data volume should be driven by a testing purpose. Scenario richness should be established before large-scale generation.
- The framework is expected to fit workflow-driven business applications best. Its applicability to highly real-time, algorithmic or deeply distributed systems should be validated separately.
- AI tools are implementation accelerators, not permanent framework dependencies. The methodology should remain portable as tools evolve.

Working thesis: Start with enough product intent to understand the problem, make scenario-based data concrete early, build rapidly, deliberately break the system, then formalise complex behaviour and specification from evidence.

